Symbol	Name	Functionality
Array(), Integer()	Conversion methods or conversion wrappers	Convert the argument to the stated type.
<<	Shovel	Used by a lot of different classes, including String, Array, and File, to append a value to the end of the existing value.
class << foo	Singleton operator	Allows access to the singleton class of object foo. Most often used with a class name to define class methods.
&.	Safe Navigation	In a method call such as foo&.bar, if the left side of the safe navigation operator is nil, allows the method call to proceed and return nil rather than throw an error.
<=>	Spaceship operator	In x <=> y, if x is less than y, return -1, if x is greater than y, return 1, if they are equal, return 0.
#{EXPRESSION}	String interpolation	Inside a double-quoted string, evaluates the expression, converts it to a string, and inserts it in the string.
x ? y : z	Ternary operator	If x is true, return y else return z
(ARGS) -> { EXPR }	Stabby Lambda	Creates a lambda equivalent to lambda { |ARGS| EXPR }.
::	Scope resolution	Looks up a constant value inside a module or class, as in Foo::Bar. If there is nothing on the left of the operator, like ::Foo, forces a lookup in the top-level global scope, which can be useful if you are doing a lookup from inside a nested set of modules.
*	Splat	In a method call, as in thing(*[1, 2, 3]) converts an array to a series of positional arguments in the method. In a method definition, converts an arbitrary number of optional arguments to an array.
**	Double Splat	In a method call, converts a hash to a set of keyword arguments in the method. In a method definition, converts an arbitrary number of keyword arguments to a hash.
&, specifically &:foo	Proc operator	In a method call, converts a proc or lambda passed as an explicit argument to the implicit block argument of the method. If the parameter decorated with the & is not a Proc, Ruby will try to convert it to one by calling the method to_proc. So, thing(&:foo) will call to_proc on the symbol :foo. In a method definition, captures the implicit block argument and converts it to a Proc that can be referred to in the method.
...	Argument Forwarding	Passes all the arguments to a given method to a new method, whether they are positional arguments, keyword arguments, or block arguments. The outer method needs to define the arguments as something like def outer(...), and the inner call has to also use the three dots: user.inner(...)